DOSSIER DE RENDU · ITÉRATION 1

Nancy Rock Vibration

Billetterie du festival : conception, maquettes et première version opérationnelle de l'API et du site

ÉQUIPE

Kamdine HASSANZADEH
Ilyass DGHISSI
Saad MOUBINE
Luca VIDIRI

MODULE

SAE « Atelier-projet de développement web 1 »
Université de Lorraine

VERSION

V1 opérationnelle
21 octobre 2026

Sommaire

Ce dossier présente le travail réalisé pendant l'itération 1 : l'analyse du sujet, les choix de conception, les maquettes des écrans, l'environnement technique, la première version de l'API et du site, et les mesures de sécurité.

1 Contexte et objectifs 3

- Le festival 3
- Fonctionnalités attendues 3
- Contraintes techniques 4

2 Organisation du projet 5

- Trello 5
- Git 5

3 Conception 6

- 3.1 Scénarios d'usage 6
- 3.2 Règles métier 7
- 3.3 Diagramme de classes 8
- 3.4 Base de données 9
- 3.5 Contrat d'API 10

4 Maquettes des écrans 11

- 4.1 Programme du festival 11
- 4.2 Détail d'une soirée 12
- 4.3 Panier et Pass 3 jours 13
- 4.4 Connexion et paiement 14
- 4.5 Billets imprimables 15
- 4.6 Backoffice : suivi des jauges 16

5 Environnement technique 17

- Docker Compose 17
- Installation 17
- Jeu de données de test 18

6 Backend réalisé (itération 1) 19

- 6.1 Architecture hexagonale 19
- 6.2 Parcours d'une requête 20
- 6.3 Endpoints disponibles 20
- 6.4 Tests unitaires 22

7 Frontend réalisé (itération 1) 23

- 7.1 Pages et parcours 23
- 7.2 Organisation du code JavaScript 25
- 7.3 Sass : architecture et bibliothèque d'outils 26
- 7.4 Grid et Flexbox : quand et pourquoi 27
- 7.5 Adaptation au terminal 28
- 7.6 Qualité HTML et vérifications 29

8 Sécurité 30

- 8.1 Injections SQL 30
- 8.2 Validation des entrées 31
- 8.3 Côté site : XSS et en-têtes HTTP 31
- 8.4 Tests d'attaque 33

9 Bilan et perspectives 34

- État d'avancement 34
- Itération 2 34
- Déploiement 34

1 Contexte et objectifs

NRV.net est l'application de billetterie du festival **Nancy Rock Vibration**. Elle permet de consulter le programme, d'acheter des billets en ligne et, pour l'organisation, de suivre le remplissage des soirées.

Le festival

- Le festival dure **2 semaines**. Chaque **soirée** a lieu à une date unique, dans un lieu donné, avec une jauge (nombre maximal de places).
- Une soirée regroupe **2 à 3 spectacles**. Chaque spectacle a **un seul style** de musique (classic rock, blues rock, métal, chanson…), une soirée peut mélanger les styles.
- Un **billet** donne accès à tous les spectacles de sa soirée. Deux tarifs : **normal** et **réduit** (étudiants, demandeurs d'emploi, personnes en situation de handicap).
- Le **Pass 3 jours** est un billet spécifique qui donne accès à **3 soirées distinctes**, avec une remise de 3 %. Il existe aussi en normal et en réduit.
- L'achat se fait uniquement en ligne, après inscription, et seulement s'il reste des places. Le billet n'est pas nominatif : il est accessible par une URL contenant un jeton aléatoire.

Fonctionnalités attendues

Réf.	Fonctionnalité du sujet	État à l'itération 1
F1	Liste des spectacles : titre, date, horaire, image	**Réalisé (API + site)**
F2	Détail d'une soirée : nom, thématique, date, lieu, tarifs, spectacles (artistes, description, style, vidéo)	**Réalisé (API + site)**
F3	Clic sur un spectacle → détail de sa soirée	**Réalisé (lien HATEOAS)**
F4 · F5	Filtrage de la liste par date et par style	**Réalisé (API + site)**
F6 · F7	Ajout au panier, état du panier et total	**Réalisé (site)**
F8 · F9	Validation en commande, paiement simulé avec contrôle et mise à jour des places	Conçu (itération 2)
F10	Billets imprimables (nom de l'acheteur, référence, date, catégorie)	Maquetté, CSS d'impression prêt
F11	Backoffice : places vendues et recette par soirée	Vue SQL et maquette
E1	Pagination de la liste des spectacles	**Réalisé (API + site)**
E2 · E4	Modification du panier, vérification des places à l'ajout	**Réalisé (site)**
E3	Backoffice : ajout de spectacles et de soirées	Non commencé

Contraintes techniques

Backend

- PHP 8 / Slim 4 dans des conteneurs Docker
- API RESTful en JSON, liens HATEOAS
- Architecture hexagonale stricte, injection de dépendances (PHP-DI)
- PDO avec requêtes préparées, validation des données
- PSR-1 et PSR-4, tests unitaires (PHPUnit)

Frontend

- HTML5 sémantique, valide W3C
- Sass/SCSS, responsive (mobile, tablette, ordinateur)
- CSS Grid et Flexbox, feuille `@media print` pour le billet
- Données chargées par `fetch` et affichées avec Handlebars

L'application doit être déployée dans des conteneurs Docker sur **docketu.iutnc.univ-lorraine.fr**.

2 Organisation du projet

Trello

Le travail est planifié par étapes sur un tableau Trello ; chaque carte correspond à un ou plusieurs commits.

Étape	Cartes
1. Conception et spécifications	scénario d'usage · diagramme de classes · BDD et schéma SQL · wireframes de la grille, de la fiche soirée · maquettes du panier, du formulaire de commande, du billet · wireframe du backoffice · mise en commun
2. Environnement	dépôt GitHub · docker compose · script SQL · script d'injection des données · structure Sass/SCSS · squelette de l'architecture hexagonale
3. Production	[BACK] arborescence et conteneur de dépendances · [BACK] entités et interfaces repository · [BACK] adaptateurs PDO · [BACK] actions Slim et routes · [FRONT] socle Sass et layouts · [FRONT] templates Handlebars et fetch · [FRONT] filtrage dynamique · [FRONT] vue détaillée d'une soirée

Git

- Dépôt du groupe : **Atelier-Projet-de-developpement-web-1**, branche de travail `production-kam`.
- Un commit par carte, dont le message reprend le nom de la carte : `2.1 docker-compose et config`, `[back] adaptateurs pdo`…
- Les commentaires du code sont courts et rédigés en français.

Historique de l'itération 1

```
2.1 docker-compose et config
2.2 seed fixtures
2.3 arborescence hexagonale
1.3 schema bdd pass 3 jours en billet
2.2 seed fixtures avec pass 3 jours
initialiser la structure sass/scss
fix volumes docker pour selinux (fedora)
fix composer : php-jwt 7 (faille de securite)
[back] entites domaine et interfaces repository
[back] adaptateurs pdo
2.2 seed : groupes en plus pour eviter les noms
de secours
[back] actions slim et routes api
[back] arborescence hexagonale et conteneur de
dependances
[back] ports de sortie dans application/port/out
tests unitaires phpunit du domaine et des cas
d'usage
preparer le deploiement docketu (ports, cors,
url de l'api)
readme : installation, donnees de test, api,
tests, deploiement
1.5 maquettes visuelles des ecrans
document de rendu iteration 1 et diagrammes uml
readme simplifie
[front] socle sass/scss et layouts
[front] templates handlebars et appels fetch
[front] composants de filtrage dynamique
[front] vue detaillee d'une soiree
[front] images locales et videos reelles dans le
seed
[front] page panier, navigation et bouton video
securite : validation des parametres et en-tetes
http
```

La documentation accompagne le code dans le dépôt : `README.md` (installation, données de test, API, tests, déploiement), `docs/arborescence.md`, `docs/maquettes/`, `docs/uml/` (sources des diagrammes) et `sql/01_schema.sql`.

3 Conception

3.1 Scénarios d'usage

Trois acteurs utilisent l'application : le **visiteur** (anonyme, il consulte), l'**acheteur** (inscrit, il commande et paie) et l'**administrateur** (il suit les jauges).

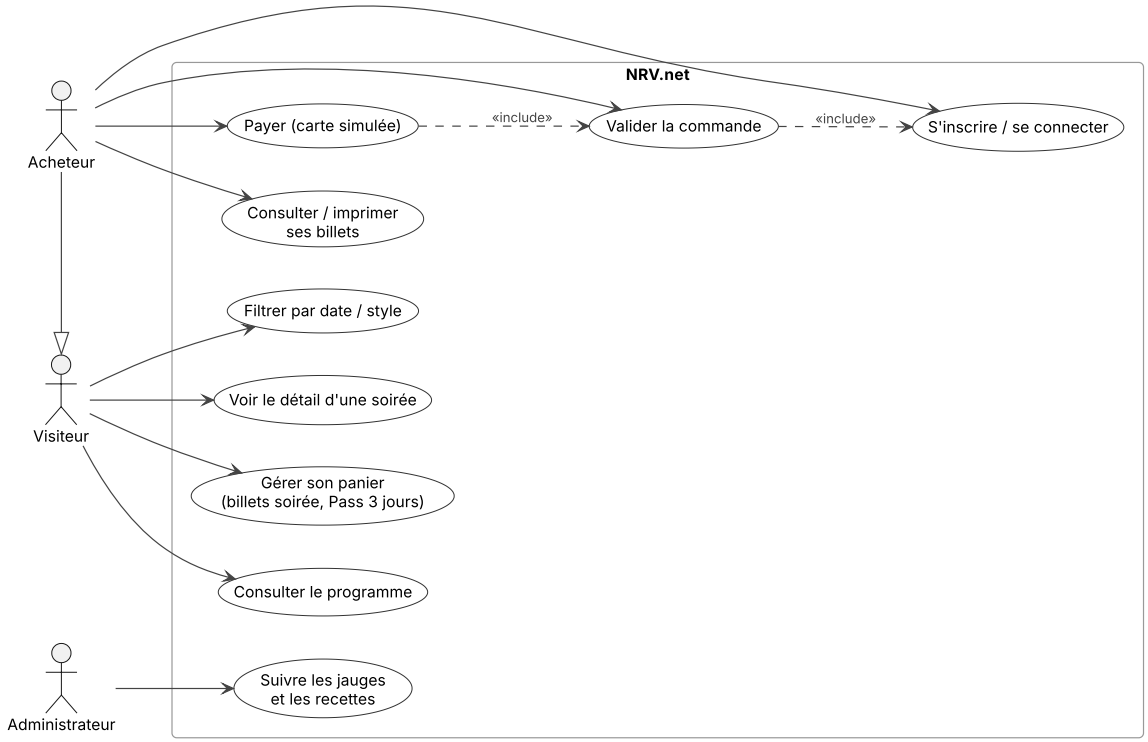


Figure 1 · Cas d'utilisation de NRV.net

Parcours A · Léa découvre le programme (visiteur, sur téléphone)

1. Elle ouvre l'accueil : la grille des spectacles s'affiche (image, titre, date, horaire), 12 par page.
2. Elle choisit le style « Métal » puis la date du 11 juin : la grille se met à jour sans recharger la page.
3. Elle touche une carte : le détail de la soirée s'ouvre (lieu, tarifs, places restantes, les 3 spectacles avec leur vidéo).

Parcours B · Marc achète des billets (acheteur, sur ordinateur)

1. Sur « Nuit métal », il ajoute 2 billets au tarif normal. Le site vérifie qu'il reste assez de places.
2. Dans le panier, il compose un **Pass 3 jours** réduit pour sa fille (11, 12 et 15 juin) : 58,20 € au lieu de 60 €.
3. Il valide le panier, se connecte (ou s'inscrit) : la commande est créée au statut « en attente ».
4. Il paie avec une carte (simulée). Le serveur vérifie les places **au moment du paiement**, les réserve et crée les billets, chacun avec un jeton unique.
5. Il imprime ses billets : une page A4 sobre, sans menu, grâce à la feuille `@media print`.

Cas d'erreur prévus : carte invalide (422), soirée devenue complète entre-temps (409, rien n'est réservé), double clic sur « Payer » (409, pas de billet en double), session expirée (401).

Parcours C · Sophie suit les ventes (administratrice)

Elle ouvre le backoffice : pour chaque soirée, la jauge, les places vendues (billets et pass), les places restantes, le taux de remplissage et la recette. Les soirées complètes ou presque complètes sont mises en évidence.

3.2 Règles métier

Règle	Comment elle est appliquée
Une soirée a 2 à 3 spectacles	l'entité `Soiree` refuse les autres cas (`NombreSpectaclesInvalideException`)
Un spectacle a un seul style	clé étrangère `spectacles.style_id` ; l'entité `Spectacle` reçoit un seul `Style`
Tarifs et jauge portés par la soirée	colonnes `tarif_normal`, `tarif_reduit`, `jauge`, `places_vendues` ; tarif réduit ≤ normal ; jauge ≤ capacité du lieu
On ne vend pas au-delà de la jauge	`Soiree::reserver()` lève `SoireeCompleteException` ou `PlacesInsuffisantesException` ; contrainte `CHECK` en base en plus
Pass 3 jours = un billet spécifique	formule `pass` en base (87,30 € / 58,20 €) ; billet de type `PASS` relié à 3 soirées distinctes ; il prend une place dans chacune
Places contrôlées au paiement	transaction : verrouillage des soirées, contrôle, réservation, création des billets ; tout est annulé au premier refus
Billet non nominatif, accessible par URL	jeton UUID v4 aléatoire unique ; le nom de l'acheteur est seulement imprimé
Montants exacts	tous les prix sont stockés en centimes (entiers), pas de flottants

3.3 Diagramme de classes

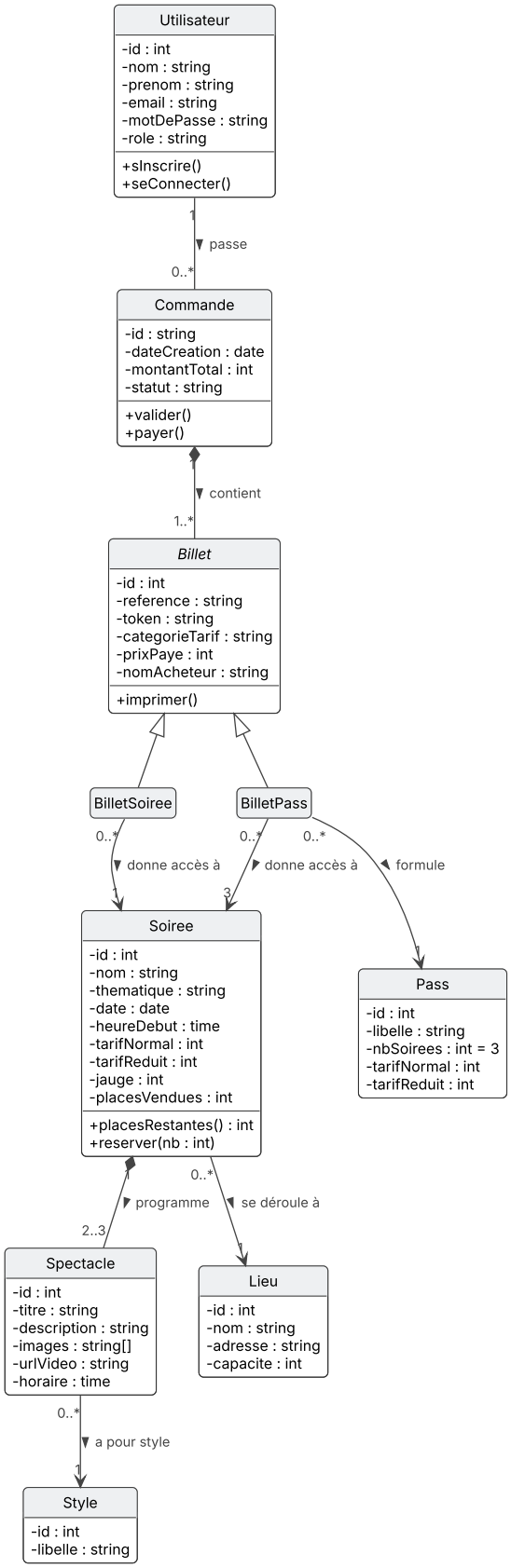


Figure 2 · Modèle du domaine

Comment le lire

- **Utilisateur → Commande → Billet** : un utilisateur passe des commandes ; une commande contient ses billets (composition, losange plein).
- **Billet** est abstrait, avec deux sortes :
  - **BilletSoiree** donne accès à **1** soirée ;
  - **BilletPass** donne accès à **3** soirées et suit une formule **Pass** (tarif normal ou réduit).
- **Soiree** porte les tarifs, la jauge et les places vendues ; elle *programme* 2 à 3 **Spectacle** (composition) et *se déroule* dans un **Lieu**.
- Chaque **Spectacle** a exactement **un Style**.

> **Correspondance en base**
>
> En base, l'héritage `Billet` est représenté par une seule table `billets` avec une colonne `type` (SOIREE ou PASS). Les 3 soirées d'un pass sont dans la table `billet_pass_soirees`.

> **Choix de modélisation**
>
> - le Pass 3 jours est un billet à part entière, et non une remise calculée sur le panier ;
> - le style est une classe, reliée à chaque spectacle par une multiplicité 1 ;
> - le panier n'est pas une classe : il vit dans le navigateur et n'est pas enregistré ;
> - le paiement simulé est une méthode de `Commande`.

3.4 Base de données

MariaDB 10.11, 14 tables InnoDB en `utf8mb4`. Le script complet est `sql/01_schema.sql` ; il est joué automatiquement au premier démarrage du conteneur.

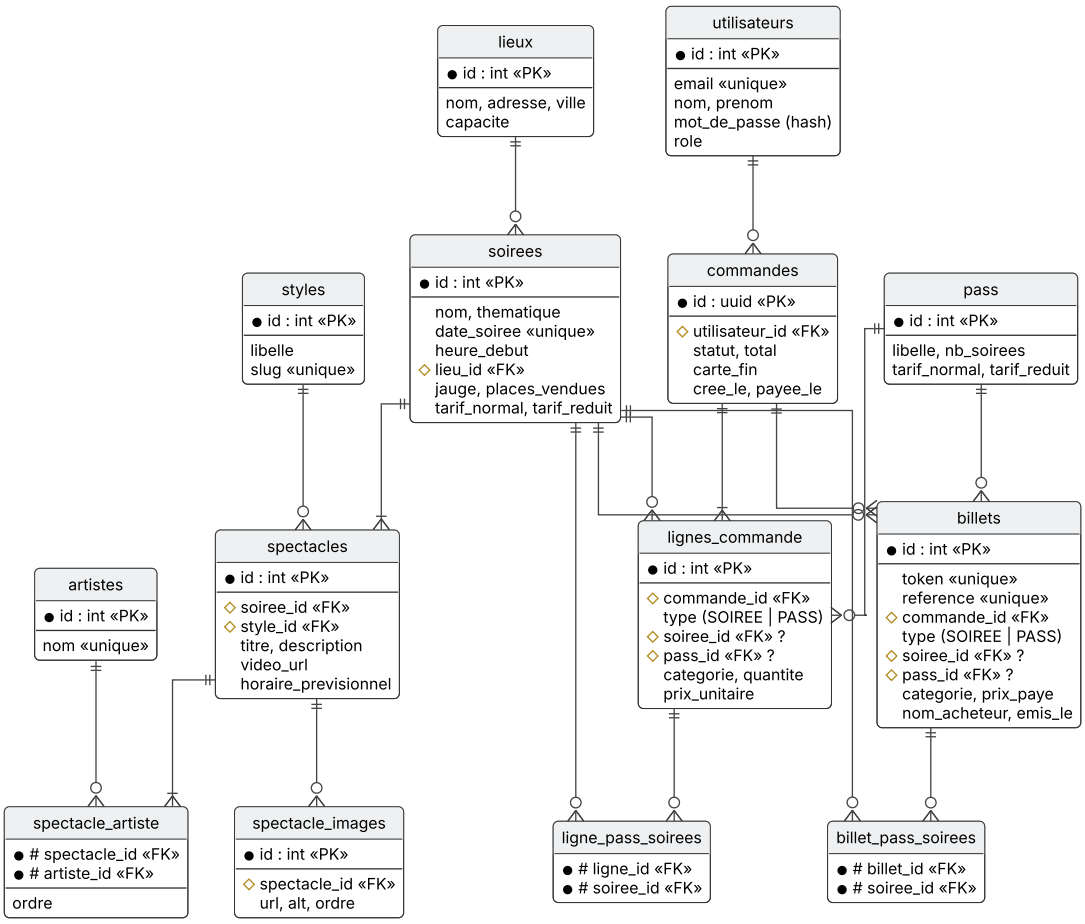


Figure 3 · Schéma relationnel (PK clé primaire, FK clé étrangère, ? = facultatif)

Tables	Rôle
`lieux`, `styles`, `artistes`	données de référence
`soirees`, `spectacles`	le programme ; `soirees.places_vendues` est le compteur de la jauge
`spectacle_artiste`, `spectacle_images`	un spectacle a un ou plusieurs artistes et une ou plusieurs images
`pass`	la formule Pass 3 jours et ses deux tarifs
`utilisateurs`, `commandes`, `lignes_commande`, `ligne_pass_soirees`	les comptes et les commandes ; une ligne = n billets identiques (soirée ou pass)
`billets`, `billet_pass_soirees`	les billets émis au paiement, avec leur jeton unique
vue `v_jauges_soirees`	pour le backoffice : jauge, vendues, restantes, taux, recette (un pass compte pour un tiers dans chacune de ses soirées)

Contraintes notables : une soirée par date (`UNIQUE`), `places_vendues ≤ jauge` , tarif réduit ≤ normal, cohérence type / soirée / pass sur les billets (`CHECK`), jeton et référence uniques.

3.5 Contrat d'API

Toutes les réponses sont en JSON. Une ressource : `{"type":"resource", "<nom>":{…}, "links":{…}}` ; une liste : `{"type":"collection", "count", "total", …, "links"}`. Les erreurs suivent le format RFC 7807 (`application/problem+json`).

Verbe	Route	Rôle	État
GET	`/spectacles`	liste paginée, filtres `date` et `style`	**Réalisé**
GET	`/spectacles/{id}`	un spectacle	**Réalisé**
GET	`/soirees/{id}`	soirée complète avec ses spectacles	**Réalisé**
GET	`/styles` , `/dates`	valeurs des filtres	**Réalisé**
GET	`/pass`	formule Pass 3 jours et ses tarifs	Itération 2
POST	`/auth/register` , `/auth/signin`	inscription, connexion (jeton JWT)	Itération 2
POST	`/commandes`	panier → commande « en attente »	Itération 2
POST	`/commandes/{id}/paiement`	paiement simulé, réservation des places, création des billets	Itération 2
GET	`/commandes` , `/commandes/{id}/billets`	mes commandes et mes billets	Itération 2
GET	`/billets/{token}`	un billet, par son jeton	Itération 2
GET	`/admin/jauges`	jauges et recettes (rôle admin)	Itération 2

Formats : montants en centimes (`3000` = 30,00 €), dates ISO (`2027-06-11`), heures `HH:MM` .

4 Maquettes des écrans

Les maquettes sont des pages HTML statiques (`docs/maquettes/`) réalisées avec la feuille de style Sass du projet et les données du jeu de test. Elles fixent la structure HTML, la mise en page responsive et les composants de chaque écran. Les écrans Programme, Soirée et Panier sont maintenant réalisés et branchés sur l'API (partie 7) ; les maquettes Connexion, Paiement, Billets et Backoffice servent de référence pour l'itération 2.

4.1 Programme du festival

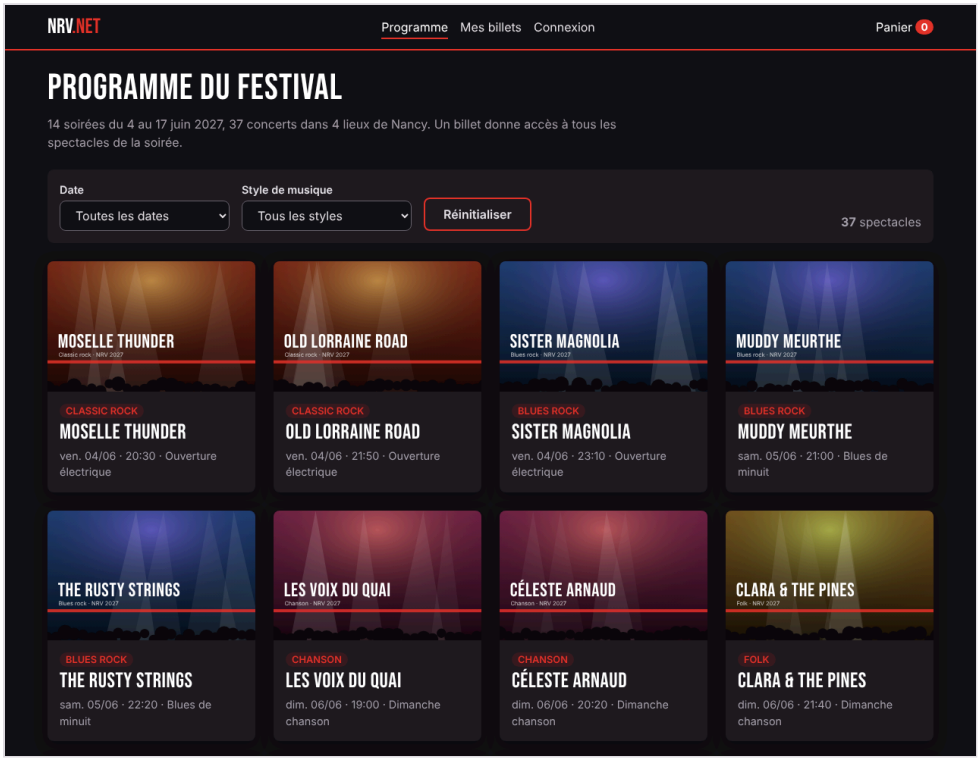


Figure 4 · Programme sur ordinateur : filtres, grille, pagination

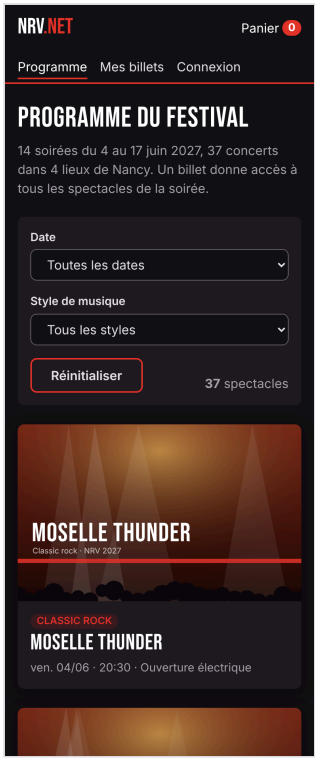


Figure 5 · Sur téléphone

Chaque carte est un lien vers la soirée (F3). Les filtres date et style se cumulent (F4, F5) et la grille se met à jour sans rechargement ; la pagination montre 12 spectacles par page (E1).

4.2 Détail d'une soirée

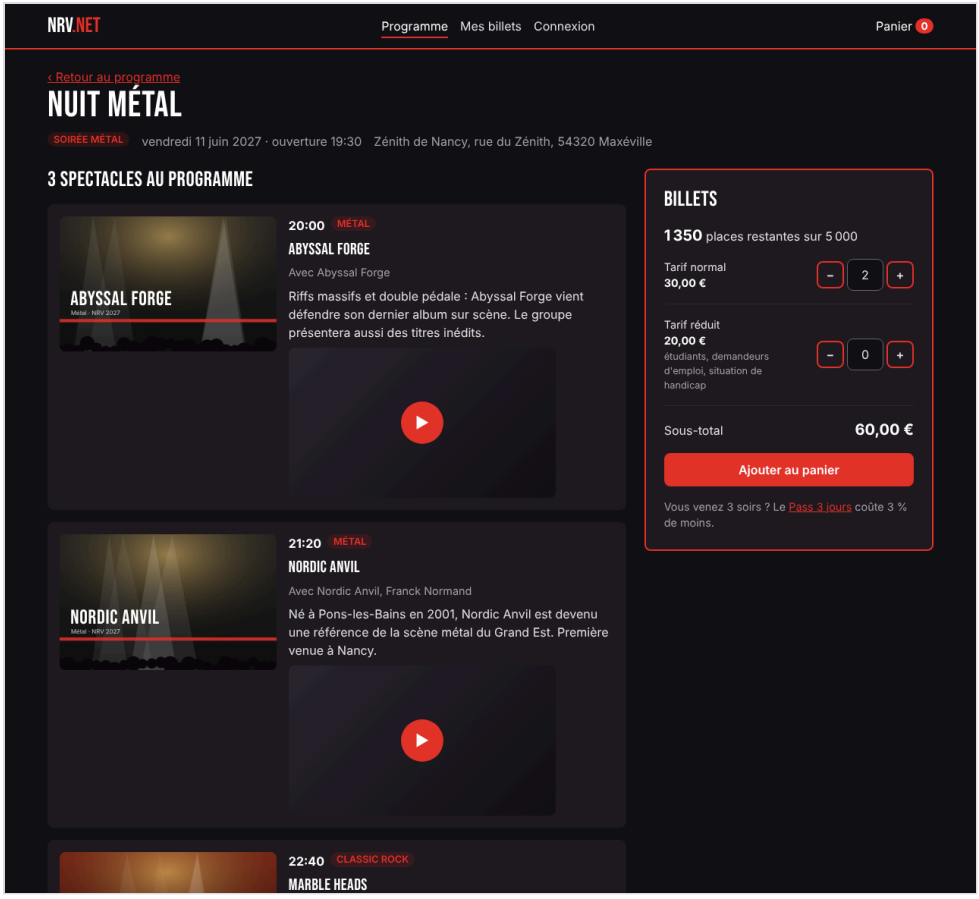


Figure 6 · Soirée « Nuit métal » : 3 spectacles et encart d'achat



Figure 7 · Sur téléphone

L'encart d'achat reste visible pendant le défilement sur ordinateur (`position: sticky`). Il affiche les places restantes, ce qui permet de bloquer un ajout impossible (E4).

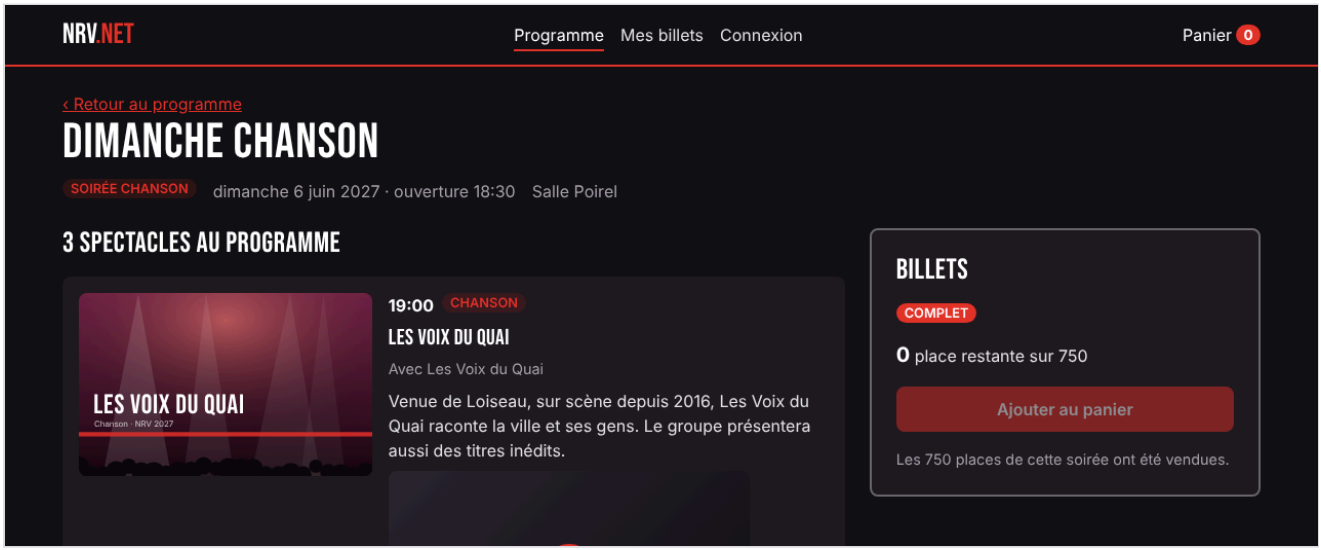


Figure 8 · Soirée complète : le bouton d'achat est désactivé

4.3 Panier et Pass 3 jours

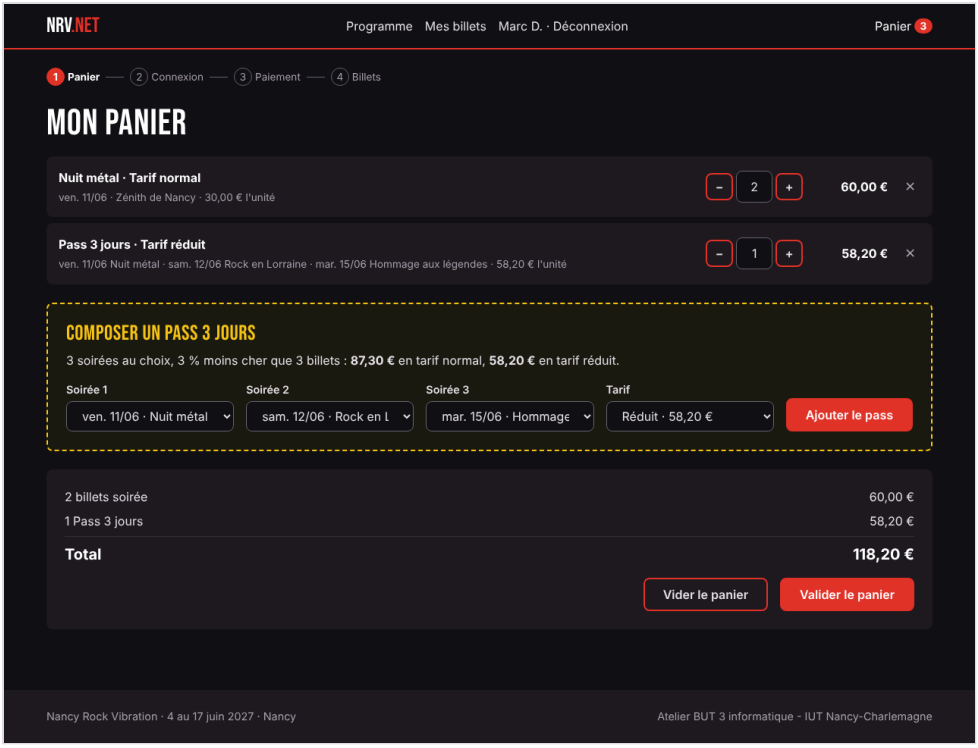


Figure 9 · Panier : billets soirée, Pass 3 jours, total



Figure 10 · Sur téléphone

Chaque ligne se modifie avec − et + (E2). L'encart « Composer un Pass 3 jours » propose 3 soirées et le tarif ; le pass devient une ligne du panier.

4.4 Connexion et paiement

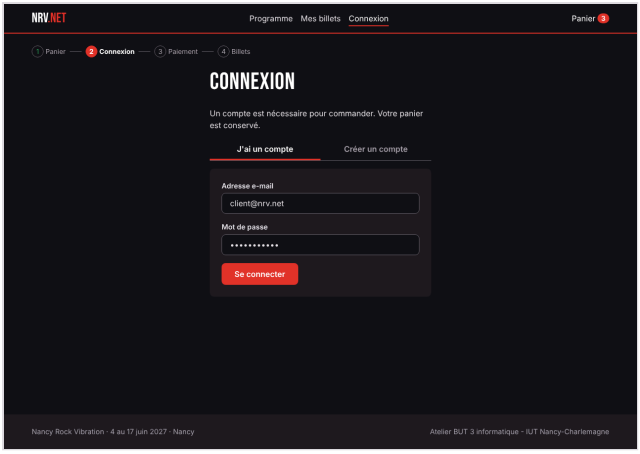


Figure 11 · Connexion (compte obligatoire pour commander)

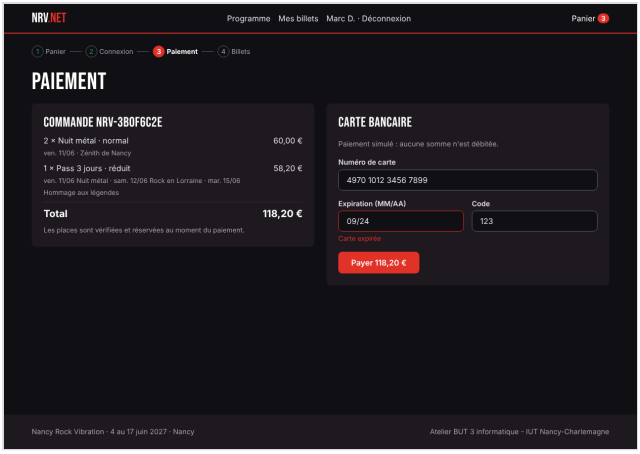


Figure 12 · Paiement : récapitulatif et carte, avec un message d'erreur

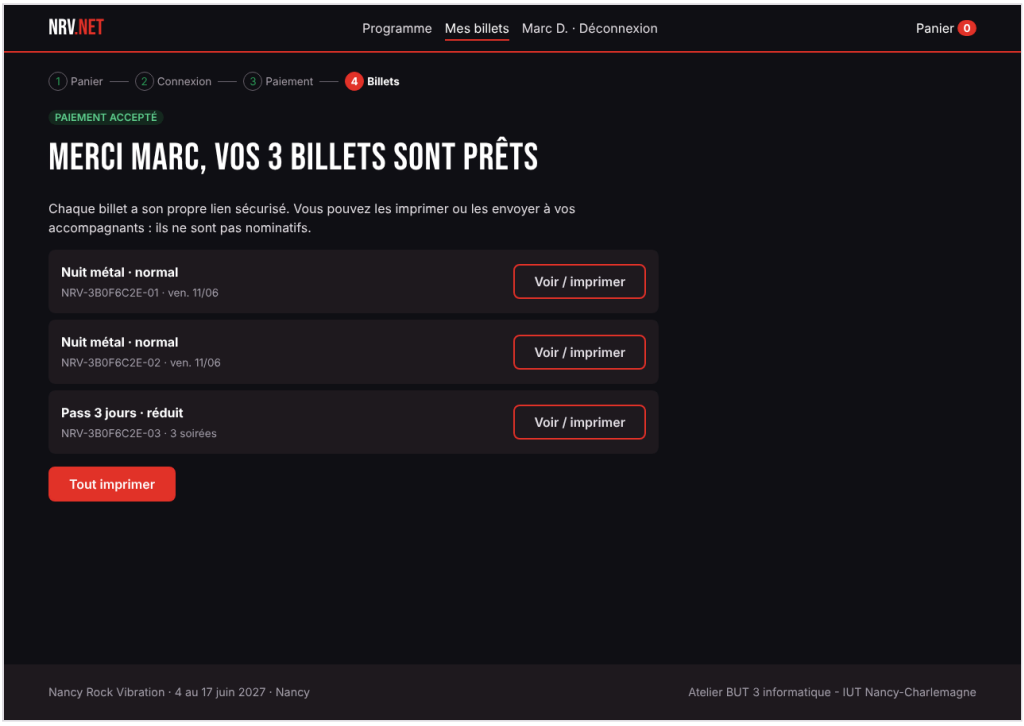


Figure 13 · Confirmation : un lien par billet

4.5 Billets imprimables

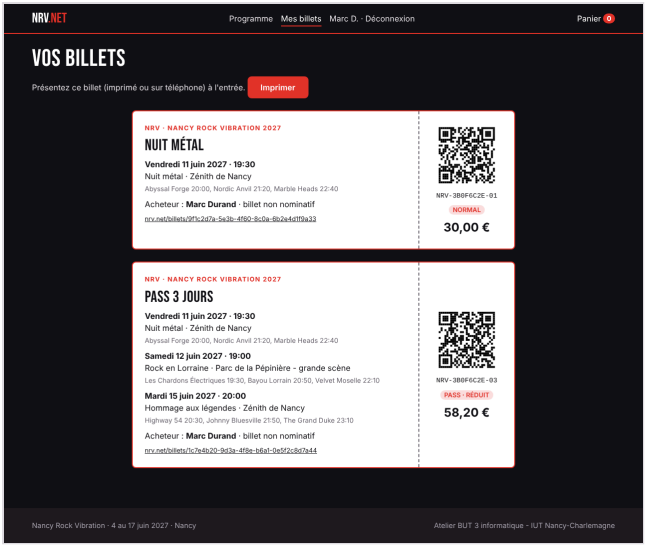


Figure 14 · Billet soirée et Pass 3 jours à l'écran

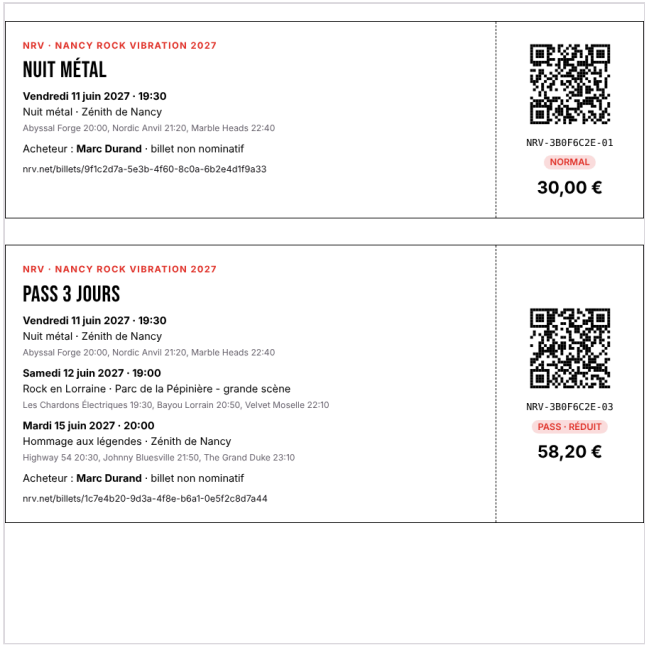


Figure 15 · Rendu à l'impression (`@media print`)

Chaque billet porte le nom de l'acheteur, la référence, la ou les soirées (date, horaire, lieu, spectacles), la catégorie et le prix (F10). À l'impression : en-tête, menu et boutons disparaissent, le billet est en noir et blanc, et il ne se coupe jamais entre deux pages.

4.6 Backoffice : suivi des jauges

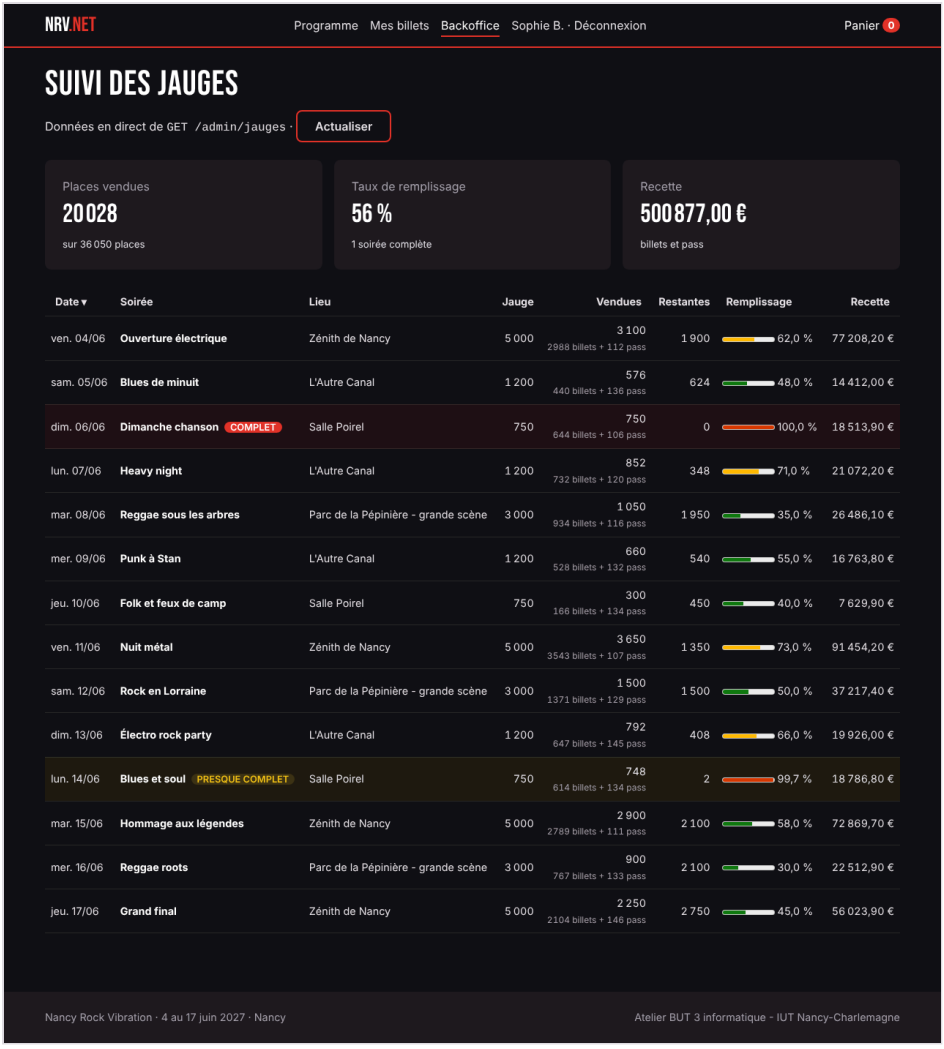


Figure 16 · Jauge et recette de chaque soirée (F11)

Trois indicateurs en haut (places vendues, taux de remplissage, recette), puis une ligne par soirée. Les soirées complètes sont en rouge, celles à plus de 90 % en jaune. Sur téléphone, le tableau défile horizontalement.

Choix de mise en page

- **CSS Grid** pour les zones en deux dimensions : grille des spectacles (`repeat(auto-fill, minmax(16rem, 1fr))` : 1 colonne sur téléphone, 4 sur ordinateur), page soirée (spectacles | achat), paiement (récapitulatif | carte), indicateurs du backoffice.
- **Flexbox** pour les alignements sur une ligne : en-tête et navigation, barre de filtres, lignes du panier, sélecteurs de quantité, étapes de la commande.
- Trois tailles d'écran (mixin Sass `mq()`) : mobile par défaut, tablette dès 600 px, ordinateur dès 1024 px.

5 Environnement technique

Docker Compose

Service	Image	Adresse	Rôle
`db`	mariadb:10.11	127.0.0.1:3307	base de données ; le schéma est joué au premier démarrage
`php`	php:8.3-fpm (Dockerfile)	interne	exécute l'API Slim ; contient Composer
`api`	nginx:1.27	localhost:8080	serveur web de l'API (tout passe par `public/index.php`)
`front`	nginx:1.27	localhost:8081	site statique (`front/public`)
`adminer`	adminer:4	localhost:8082	interface pour consulter la base
`sass`	node:20 (profil dev)	—	recompile le Sass à chaque sauvegarde

Les ports se changent dans le fichier `.env` (utile sur un serveur partagé). Sur Fedora, les volumes ont l'option `:z` pour SELinux.

Installation

```
git clone https://github.com/Kamdinee/Atelier-Projet-de-developpement-web-1.git
cd Atelier-Projet-de-developpement-web-1
git checkout production-kam
cp .env.example .env

docker compose up -d --build
docker compose exec php composer install
docker compose exec php composer seed     # jeu de donnees de test
docker compose exec php composer test     # tests unitaires
```

Jeu de données de test

Le script `api/bin/seed.php` (FakerPHP, graine fixe : les données sont toujours les mêmes) remplit la base avec :

- 4 lieux à Nancy, 8 styles, 14 soirées du 4 au 17 juin 2027, 37 spectacles, 43 artistes ;
- une photo de concert par spectacle (Unsplash, Pexels, Pixabay), servie par le site (`img/spectacles/`) et préparée par `front/tools/photos.py` , et une vraie vidéo YouTube par style ;
- la formule Pass 3 jours (87,30 € / 58,20 €) ;
- 42 utilisateurs, environ 5 600 commandes payées, 18 200 billets soirée et 600 Pass 3 jours, soit environ 503 000 € de recette.

Cas de test remarquables	Id	Situation
Dimanche chanson (6 juin, Salle Poirel)	3	**complète** : tout achat doit être refusé
Blues et soul (14 juin, Salle Poirel)	11	**2 places restantes** : 2 billets passent, 3 sont refusés
Nuit métal (11 juin, Zénith)	8	3 spectacles, grande jauge

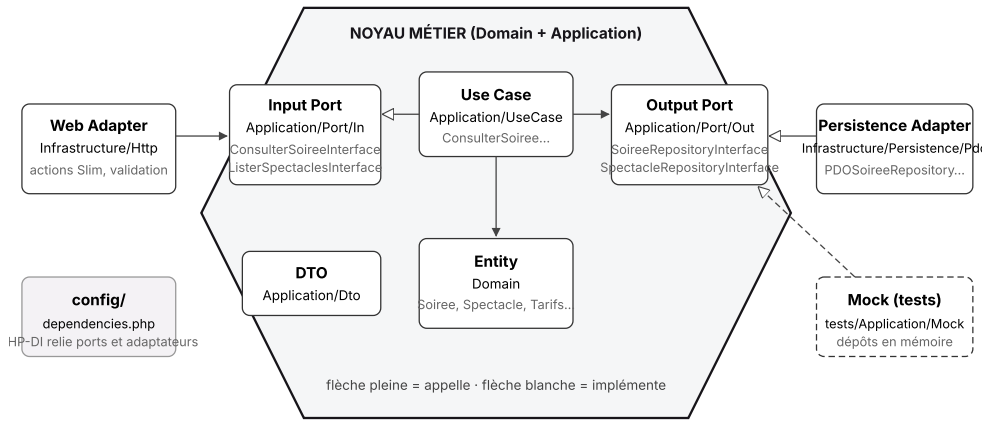
Compte	E-mail	Mot de passe
Administratrice	admin@nrv.net	admin12345
Client	client@nrv.net	client12345
40 clients fictifs	voir la table `utilisateurs`	motdepasse1

6 Backend réalisé (itération 1)

L'itération 1 livre la partie « consultation » de l'API : programme, filtres, pagination, détail des soirées. Le noyau métier est complet pour ces fonctionnalités et couvert par 49 tests unitaires.

6.1 Architecture hexagonale

Le code suit le schéma vu en cours : un **noyau métier** (entités, cas d'usage, ports) qui ne dépend de rien, entouré d'**adaptateurs**. Les ports sont des **interfaces** ; les cas d'usage sont des **services**.



> **Inversion de dépendances**
>
> Le cas d'usage `ConsulterSoiree` demande une soirée à une **interface** (`SoireeRepositoryInterface`) sans savoir qu'il y a PDO derrière. C'est `config/dependencies.php` qui branche l'implémentation PDO (inversion de dépendances). Dans les tests, on branche un faux dépôt en mémoire à la place, sans rien changer au cas d'usage.

Couche	Dossier	Contient · dépend de
Domaine	`src/Domain`	entités (`Soiree` , `Spectacle` , `Lieu` , `Style`), value objects (`Tarifs` , `Image` , `CategorieTarif` , `CriteresSpectacles` ...), exceptions métier · **ne dépend de rien** (PHP natif seulement)
Application	`src/Application`	ports d'entrée et de sortie (interfaces), cas d'usage (services), DTO · dépend du domaine seulement
Adaptateur d'entrée	`src/Infrastructure/Http`	actions Slim, validation des paramètres, erreurs JSON, CORS, liens HATEOAS · appelle les ports d'entrée
Adaptateur de sortie	`src/Infrastructure/Persistence/Pdo`	dépôts PDO · implémentent les ports de sortie

6.2 Parcours d'une requête

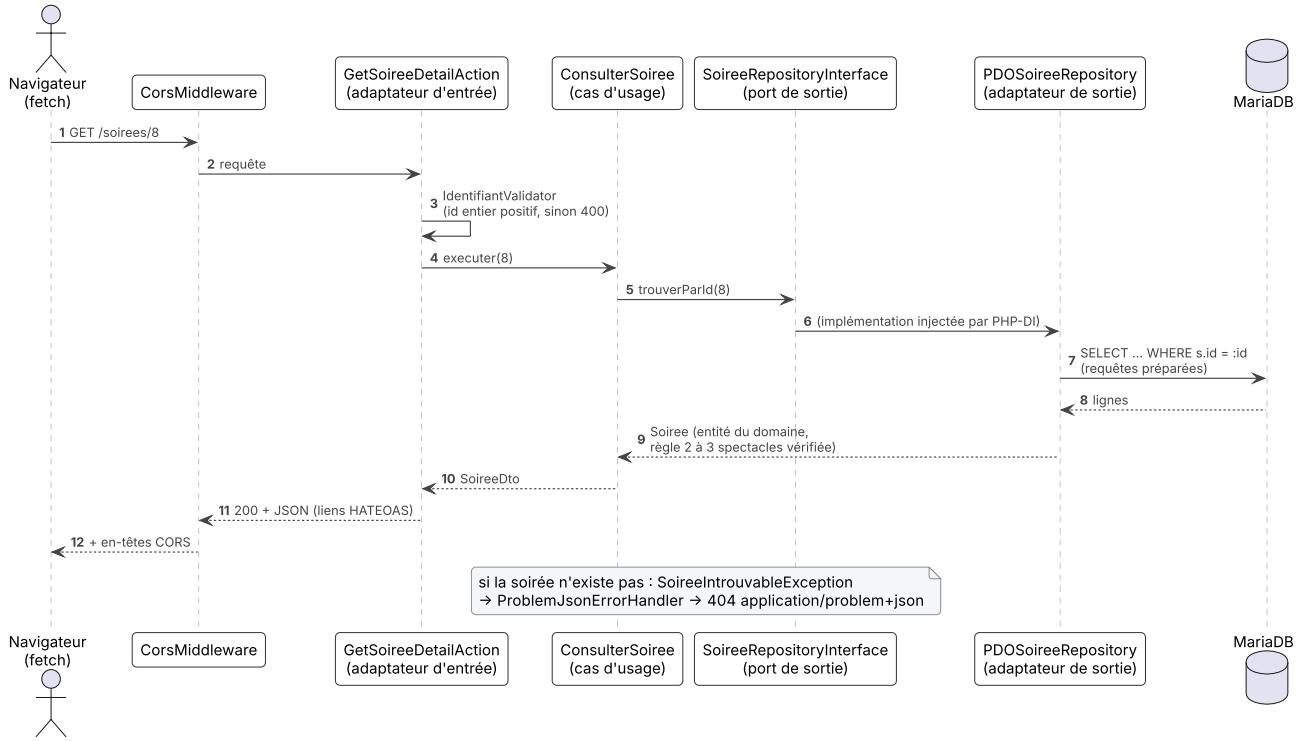


Figure 17 · Ce qui se passe pour `GET /soirees/8`

Les middlewares, dans l'ordre

Slim exécute en premier le middleware ajouté en dernier. Dans `config/middlewares.php` :

1. **CorsMiddleware** : ajoute les en-têtes `Access-Control-*` pour les origines autorisées, répond 204 aux requêtes `OPTIONS`. Il est à l'extérieur pour que même les réponses d'erreur aient les en-têtes CORS.
2. **ErrorMiddleware** avec `ProblemJsonErrorHandler` : transforme toute exception en réponse JSON avec le bon code HTTP.
3. **RoutingMiddleware** : trouve l'action à appeler.
4. **BodyParsingMiddleware** : décode le JSON des requêtes POST (utile à l'itération 2).

6.3 Endpoints disponibles

Route	Paramètres	Action Slim
`GET /`	—	`AccueilAction`
`GET /spectacles`	`date` (AAAA-MM-JJ), `style` (slug), `page` (≥ 1), `size` (1 à 50), tous facultatifs et cumulables	`GetSpectaclesAction`
`GET /spectacles/{id}`	id entier positif	`GetSpectacleAction`
`GET /soirees/{id}`	id entier positif	`GetSoireeDetailAction`
`GET /styles`	—	`GetStylesAction`
`GET /dates`	—	`GetDatesAction`

GET /spectacles?style=metal&size=1


```
{
  "type": "collection",
  "count": 1, "total": 7, "page": 1, "size":
1,
  "filtres": { "date": null, "style": "metal"
},
  "spectacles": [ {
    "spectacle": {
      "id": 9, "titre": "Fer Rouge",
      "date": "2027-06-07", "horaire":
"20:30",
      "image": "img/spectacles/metal-1.webp",
      "style": { "slug": "metal",
                 "libelle": "Métal" } },
    "links": {
      "self":   { "href": "/spectacles/9" },
      "soiree": { "href": "/soirees/4" } } }
],
  "links": {
    "self":  { "href": "/spectacles?
style=metal&page=1&size=1" },
    "first": { "href": "…page=1…" },
    "next":  { "href": "…page=2…" },
    "last":  { "href": "…page=7…" } }
}
```


GET /soirees/8 (extrait)


```
{
  "type": "resource",
  "soiree": {
    "id": 8, "nom": "Nuit métal",
    "thematique": "Soirée métal",
    "date": "2027-06-11", "heure_debut":
"19:30",
    "lieu": { "id": 2, "nom": "Zénith de
Nancy",
              "adresse": "rue du Zénith, …" },
    "tarifs": { "normal": 3000, "reduit": 2000
},
    "jauge": 5000, "places_restantes": 1350,
    "complete": false,
    "spectacles": [ {
      "id": 19, "titre": "Abyssal Forge",
      "artistes": [ "Abyssal Forge" ],
      "description": "Riffs massifs…",
      "style": { "slug": "metal", … },
      "horaire": "20:00",
      "images": [ { "url": "…", "alt": "…" }
],
      "video": "https://www.youtube-
nocookie.com/embed/…"
    }, … ]
  },
  "links": { "self": { "href": "/soirees/8" },
    "spectacles": { "href": "/spectacles?
date=2027-06-11" } }
}
```


Erreurs

```
GET /spectacles?date=2027-02-31&page=0

400 application/problem+json
{
  "type": "https://nrv.net/erreurs/requete-
invalide",
  "title": "Requête invalide",
  "status": 400,
  "errors": {
    "date": "format attendu : AAAA-MM-JJ",
    "page": "entier superieur ou egal a 1
attendu"
  }
}
```

Code	Cas
400	paramètre invalide (date impossible, style mal formé, id non numérique)
404	soirée, spectacle ou route inconnus
405	méthode non autorisée (en-tête `Allow`)
409	places insuffisantes (itération 2)
422	règle métier non respectée
500	erreur interne, sans aucun détail en production

La sécurité (injections, validation, en-têtes) est détaillée dans la partie 8.

6.4 Tests unitaires

49 tests, 86 assertions, tous verts (PHPUnit 11). Ils testent le noyau métier sans base de données ni Docker.

tests/Domain (entités et value objects)

- soirée de 2 ou 3 spectacles acceptée, 1 ou 4 refusée
- spectacles triés par horaire, spectacle d'une autre soirée refusé
- réservation : la jauge baisse, refus si plus de places que restantes, refus si complète, refus de 0 place
- jauge ≤ capacité du lieu, places vendues ≤ jauge
- tarifs : réduit ≤ normal, pas de négatif
- un seul style par spectacle, artiste obligatoire
- pagination : décalage, nombre de pages

tests/Application (cas d'usage)

- filtres par style, par date, combinés
- pagination, contenu des DTO
- date mal formée refusée
- soirée et spectacle introuvables → exception
- soirée complète signalée
- listes des filtres triées

> **Mocks des ports de sortie**
>
> Les cas d'usage sont testés avec `InMemorySoireeRepository` et `InMemorySpectacleRepository`, qui implémentent les mêmes interfaces que les dépôts PDO.

Lancer les tests : `docker compose exec php composer test` (ou `vendor/bin/phpunit --testdox` pour le détail de chaque test).

Contrôle supplémentaire : en cassant volontairement la règle de jauge dans `Soiree::reserver()`, le test correspondant échoue bien.

7 Frontend réalisé (itération 1)

Le site est un ensemble de pages HTML statiques qui interrogent l'API avec `fetch` et affichent les réponses avec des templates Handlebars. Il n'utilise aucun framework : du JavaScript natif en modules ES, une feuille Sass compilée et une feuille d'impression.

7.1 Pages et parcours

Page	Contenu	Fonctionnalités
`index.html`	programme : filtres date et style cumulables, compteur de résultats, grille de cartes, pagination ; les filtres et la page sont gardés dans l'URL (lien partageable, bouton « retour » du navigateur)	F1, F3, F4, F5, E1
`soiree.html?id=…`	détail d'une soirée : thématique, date, lieu, spectacles (horaire, style, artistes, description, galerie d'images, vidéo), encart d'achat avec places restantes et sous-total	F2, F6, E4
`panier.html`	lignes du panier modifiables (− / + / supprimer), total, vider le panier ; la validation de la commande ouvre à l'itération 2	F7, E2, E4

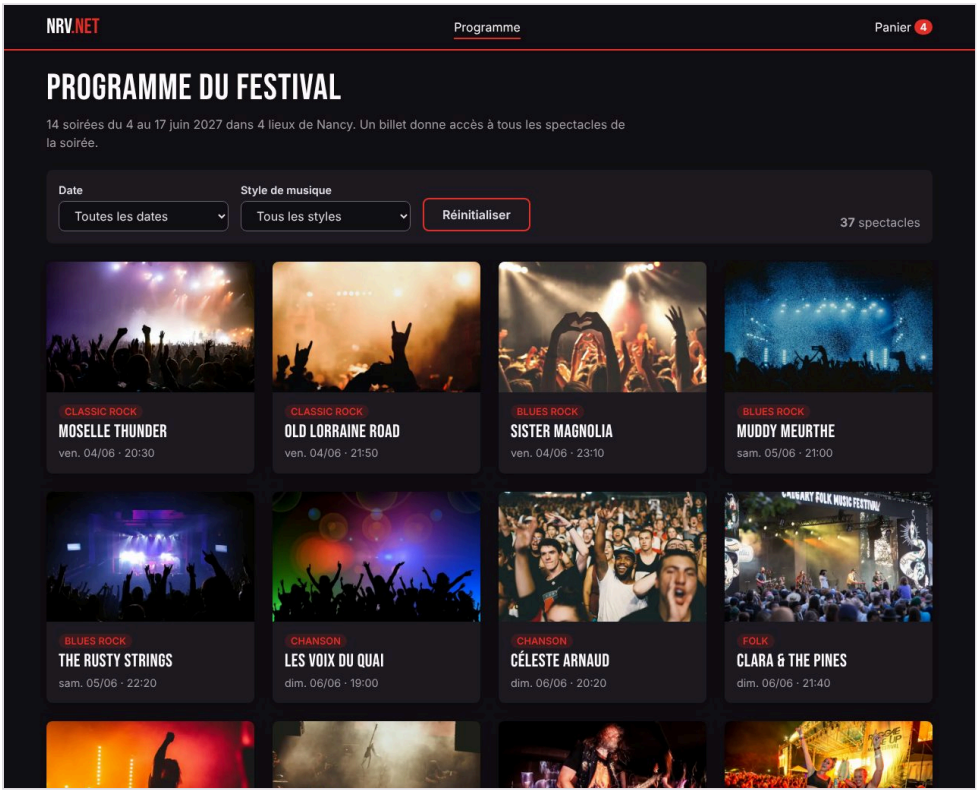


Figure 18 · Programme sur ordinateur (données réelles de l'API)

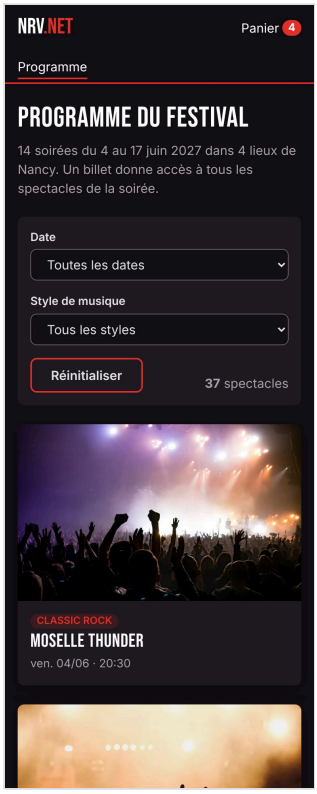


Figure 19 · Sur téléphone

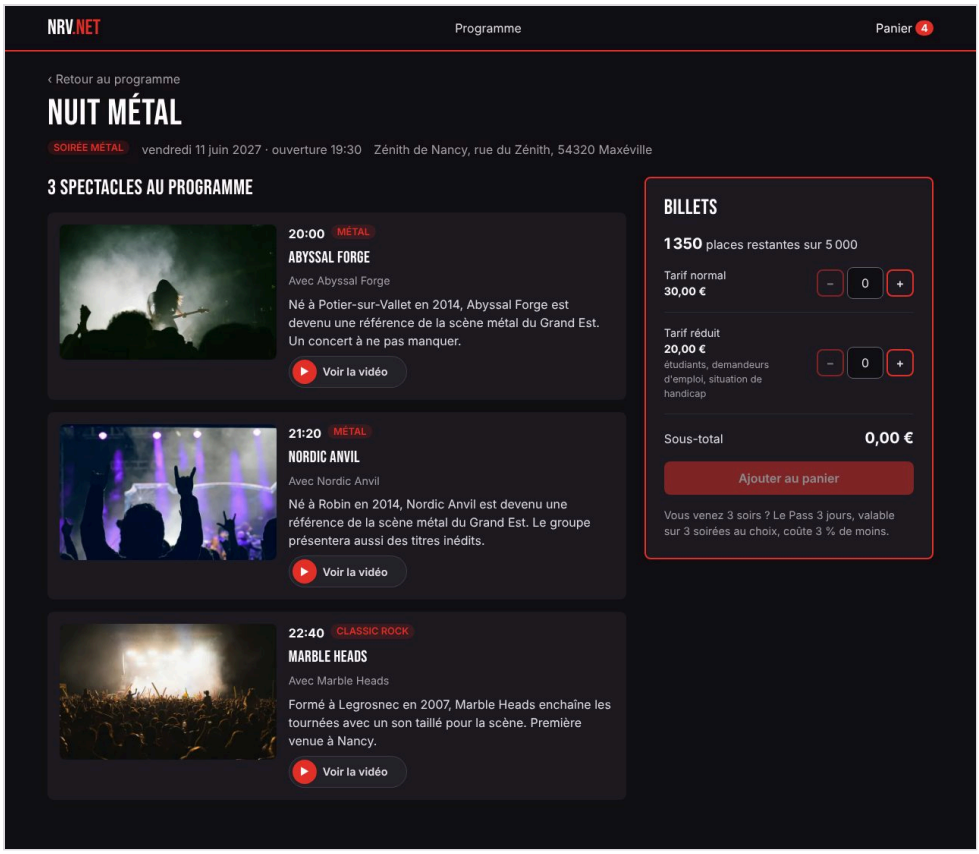


Figure 20 · Soirée « Nuit métal » : 3 spectacles, vidéo chargée au clic, encart d'achat

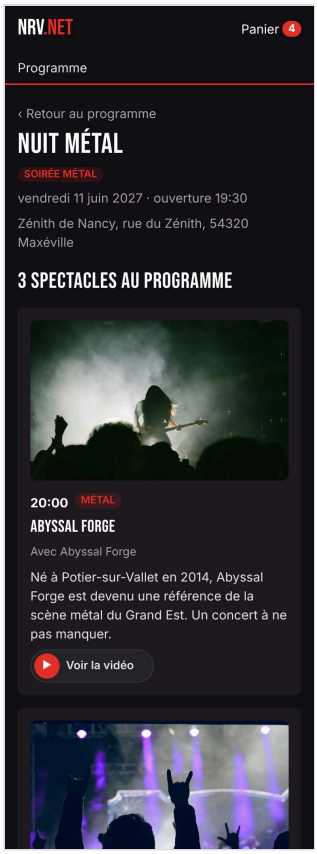


Figure 21 · Sur téléphone

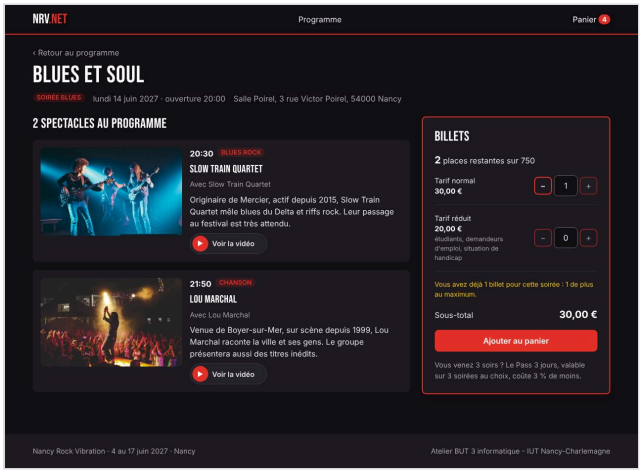


Figure 22 · Soirée à 2 places : le + se bloque et un message explique pourquoi (E4)

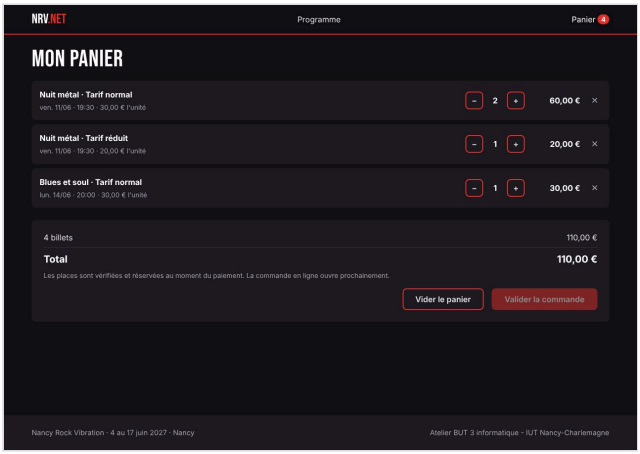


Figure 23 · Panier : quantités modifiables et total (F7, E2)

Écarts avec les maquettes

La composition du Pass 3 jours dans le panier et les écrans Connexion, Paiement, Billets et Backoffice dépendent des routes de commande et d'authentification : ils sont prévus pour l'itération 2. Les liens correspondants ont été retirés de la navigation pour ne jamais mener à une page vide.

7.2 Organisation du code JavaScript

Module	Rôle
`config.js`	adresse de l'API, déduite de celle du serveur (même nom d'hôte, port 8080) : rien à changer pour le déploiement
`api.js`	un seul point d'appel à `fetch` ; transforme une erreur `application/problem+json` en `ErreurApi` (code HTTP, titre, détails) ; lit l'id dans un lien HATEOAS
`templates.js`	charge les fichiers `templates/*.hbs`, les compile une seule fois (cache) et enregistre les helpers de formatage
`format.js`	prix en centimes → « 30,00 € », dates et horaires en français (`Intl`)
`ui.js`	chargement, messages d'erreur avec bouton « Réessayer », notification après un ajout au panier
`programme.js` · `soiree.js` · `panier-page.js`	le code propre à chaque page
`panier.js`	le panier, gardé dans le `localStorage` ; il est synchronisé entre onglets grâce à l'événement `storage`

Du clic à l'affichage : un filtre sur le programme

1. Le choix d'un style met à jour l'URL (`history.pushState`) sans recharger la page.
2. La requête précédente, si elle n'est pas finie, est annulée (`AbortController`) : un résultat ancien ne peut pas écraser le nouveau.
3. `GET /spectacles?style=metal&page=1` renvoie la page de résultats et ses liens `next` / `prev`.
4. Chaque carte pointe vers sa soirée grâce au lien HATEOAS `links.soiree` ; la pagination n'affiche « suivant » que si l'API fournit `links.next`.
5. Le template `spectacles.hbs` produit le HTML, inséré dans la grille ; le compteur annonce le résultat aux lecteurs d'écran (`aria-live`).

```
{{!-- templates/carte-spectacle.hbs : toute la carte mène à la soirée (F3) --}}
<article class="carte-spectacle">
  <img class="carte-spectacle__image" src="{{image}}" alt="{{titre}} sur scène"
       width="800" height="500" loading="lazy">
  <div class="carte-spectacle__corps">
    <span class="etiquette">{{style.libelle}}</span>
    <h3 class="carte-spectacle__titre">
      <a class="carte-spectacle__lien" href="soiree.html?id={{soireeId}}">{{titre}}</a></h3>
    <p class="carte-spectacle__infos"><time datetime="{{date}}T{{horaire}}">{{dateCourte date}} ·
{{horaire}}</time></p>
  </div>
</article>
```

Les cas d'erreur sont tous gérés à l'écran : API injoignable, filtre invalide dans l'URL (400), soirée inconnue (404), aucun résultat, soirée complète (bouton d'achat désactivé).

7.3 Sass : architecture et bibliothèque d'outils

```
src/scss/
├── abstracts/  variables, mixins
├── base/       reset, typographie
├── layout/     conteneur, en-tête, pied
├── components/ boutons, cartes, filtres…
├── pages/      soirée, panier, billet…
├── main.scss   → css/main.css
└── print.scss  → css/print.css
```

`abstracts/` ne produit aucun CSS : il ne contient que des outils. Les fichiers sont reliés avec `@use` (modules Sass) et les classes suivent la convention BEM : `.carte-spectacle__titre`, `.etiquette--complet`.

```scss
// abstracts/_variables.scss
$breakpoints: ('tablette': 600px, 'bureau': 1024px);

// abstracts/_mixins.scss
@mixin mq($nom) {
  @if not map.has-key($breakpoints, $nom) {
    @error "breakpoint inconnu : #{$nom}";
  }
  @media (min-width: map.get($breakpoints, $nom)) {
    @content;
  }
}

@mixin flex($justify: flex-start, $align: center,
            $gap: $espace-m, $wrap: nowrap) {
  display: flex;
  justify-content: $justify;
  align-items: $align;
  gap: $gap;
  flex-wrap: $wrap;
}
```

La bibliothèque d'outils (`abstracts/`)

Outil	Usage
variables	couleurs par rôle (`$couleur-accent` , `$couleur-surface` …), polices, échelle d'espacements (`$espace-xs` à `$espace-xl`), rayon, ombre, largeur maximale, map des points de rupture
`@mixin mq($nom)`	media query « mobile d'abord » à partir d'un nom ; une faute de frappe arrête la compilation (`@error`)
`@mixin flex(…)`	une ligne flex en une instruction, avec des valeurs par défaut : `@include flex(space-between, center, $espace-m, wrap)`
`@mixin conteneur`	largeur maximale centrée avec marges intérieures
`@mixin focus-visible`	contour jaune visible au clavier seulement, appliqué à tous les éléments interactifs
`@mixin visuellement-cache`	texte lu par les lecteurs d'écran mais invisible (« Page 2 » dans la pagination, numéro des photos de la galerie)

Sélecteurs avancés utilisés

Sélecteur	Pourquoi
`.carte-spectacle__lien::after { inset: 0 }`	le lien du titre couvre toute la carte : un seul lien par carte, donc une seule tabulation au clavier
`&:hover, &:focus-within`	la carte se soulève aussi quand son lien a le focus clavier
`.spectacle-detail { & + & { … } }`	espace seulement entre deux spectacles qui se suivent
`[aria-pressed='true']`, `[aria-current='page']`, `[aria-invalid='true']`	le style suit l'état d'accessibilité : miniature choisie, page courante, champ en erreur ; pas de classe d'état à maintenir en double
`> span[aria-hidden='true']`	la pastille « lecture » du bouton vidéo, distincte du texte lu par le lecteur d'écran
`li + li::after` avec `counter()`	étapes de commande numérotées et reliées par un trait, sans balisage en plus
`::-webkit-inner-spin-button`	retire les flèches natives des champs nombre, remplacées par les boutons – / +

7.4 Grid et Flexbox : quand et pourquoi

Règle suivie : **Grid** quand la mise en page est en deux dimensions ou quand des colonnes doivent s'aligner, **Flexbox** pour aligner des éléments sur une ligne qui peut passer à la ligne.

Composant	Choix	Justification et adaptation
Grille des spectacles	Grid	`repeat(auto-fill, minmax(16rem, 1fr))` : le nombre de colonnes dépend de la place (1 sur téléphone, 2 sur tablette, 4 sur ordinateur) sans aucune media query, et toutes les cartes d'une ligne ont la même hauteur
Page soirée	Grid	une colonne sur mobile ; `2fr 1fr` sur ordinateur (spectacles | achat), l'en-tête occupe toute la largeur (`grid-column: 1 / -1`)
Fiche d'un spectacle	Grid	image au-dessus du texte, puis côte à côte dès la tablette (`minmax(12rem, 2fr) 3fr`)
Billet, paiement, backoffice	Grid	zones fixes (billet | talon, récapitulatif | carte, indicateurs en 3 colonnes)
En-tête et navigation	Flexbox	logo | liens | panier avec `space-between` ; passe à la ligne si l'écran est étroit
Barre de filtres	Flexbox	`flex: 1 1 12rem` : les champs s'empilent sur téléphone et restent en ligne sur ordinateur ; le compteur est poussé à droite (`margin-left: auto`)
Lignes du panier, encart d'achat	Flexbox	libellé | quantité | prix sur une ligne ; sur téléphone le libellé prend toute la largeur (`flex-basis: 100%`) et les commandes passent dessous

Composant	Choix	Justification et adaptation
Sélecteur – [n] +, pagination, infos de la soirée	Flexbox	alignement vertical centré, espacement par `gap`

7.5 Adaptation au terminal

Aspect	Ce qui est fait
Images	une photo par spectacle, recadrée en 16/10 et enregistrée en WebP en deux tailles (640 et 1200 px, environ 30 et 70 Ko) ; `srcset` et `sizes` : le navigateur charge la petite version sur un écran classique et la grande sur un écran haute densité ; `width` / `height` dans le HTML et `aspect-ratio: 16 / 10` pour réserver la place (pas de saut de page au chargement) ; `object-fit: cover` ; `loading="lazy"` ; le lecteur YouTube n'est chargé qu'au clic
Mise en page	mobile d'abord, deux points de rupture (600 px et 1024 px) ; encart d'achat collant (`position: sticky`) seulement sur ordinateur ; aucun défilement horizontal à 390 px (vérifié)
Typographie	tailles en `rem` (respectent le réglage du navigateur) ; titre principal de 2,25 rem sur mobile à 3 rem sur ordinateur ; polices Inter et Bebas Neue avec polices système de secours
Interaction	zones tactiles d'au moins 36 px ; états `:hover` et `:focus-visible` distincts ; clavier numérique sur mobile (`inputmode="numeric"`) ; animations coupées si l'utilisateur le demande (`prefers-reduced-motion`) ; lien « Aller au contenu »
Impression	`print.css` (`media="print"`) : menus et boutons masqués, noir et blanc, billet jamais coupé entre deux pages

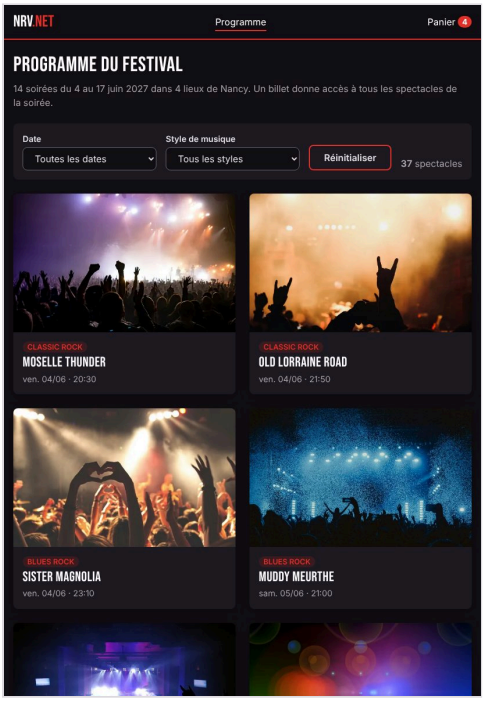


Figure 24 · Tablette (820 px) : 2 colonnes

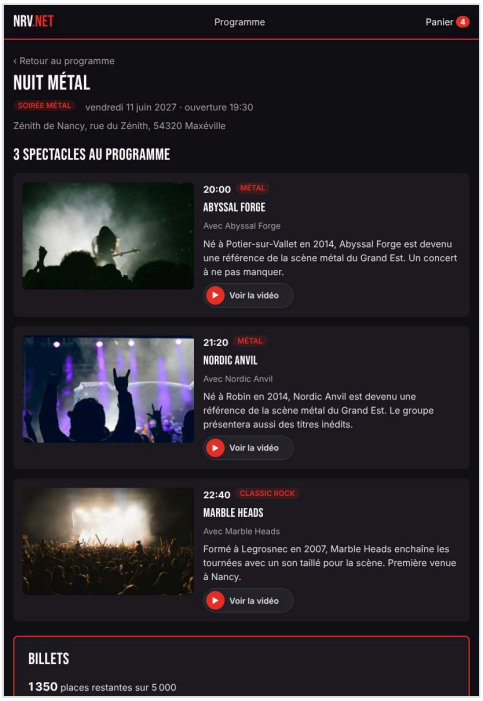


Figure 25 · Soirée sur tablette : image à côté du texte

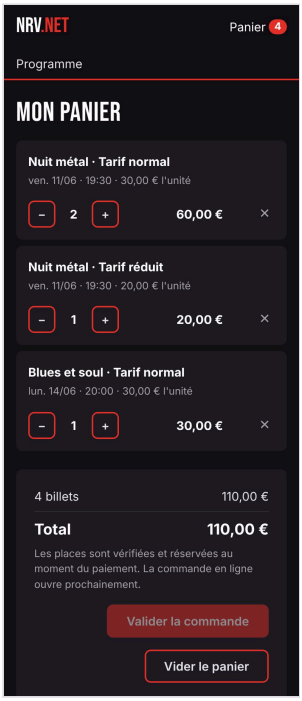


Figure 26 · Panier sur téléphone

7.6 Qualité HTML et vérifications

- **HTML valide W3C** : 0 erreur et 0 avertissement au validateur officiel (Nu Html Checker) sur les 3 pages et sur 9 états rendus par le JavaScript (programme filtré, aucun résultat, soirée, soirée complète, erreurs, panier plein et vide).
- **Sémantique** : `header`, `nav`, `main`, `footer`, une liste `ul` de cartes `article`, `time datetime` pour les dates, `output` pour les quantités calculées, `label` sur chaque champ, un seul `h1` par page et une hiérarchie de titres continue.
- **Accessibilité** : zones mises à jour annoncées (`aria-live`, `aria-busy`), boutons − / + et pagination nommés pour les lecteurs d'écran, focus visible, contraste du texte sur fond sombre.
- **Tests dans un vrai navigateur** (Playwright, derrière le serveur nginx du projet) : **46 vérifications, toutes réussies** : F1 à F7, E1, E2, E4, retour du navigateur, erreurs 400 et 404, liens de navigation, images, pas de débordement sur téléphone, aucune erreur JavaScript et aucune violation de la politique de sécurité (CSP).

8 Sécurité

Le site et l'API sont protégés contre les attaques courantes du web : injection SQL, injection de HTML ou de script (XSS), intégration dans un site piège (clickjacking) et appels depuis un autre site. Chaque protection a été vérifiée par des tests d'attaque, présentés en 8.4.

8.1 Injections SQL

Une injection SQL consiste à glisser du code SQL dans un paramètre (par exemple `?style=x' OR 1=1 --`) pour qu'il soit exécuté par la base. L'API s'en protège à deux niveaux.

Requêtes préparées partout

- La requête SQL est envoyée à MariaDB **avant** les valeurs, avec des marqueurs (`:style`, `?`). Les valeurs arrivent ensuite séparément : elles sont toujours traitées comme des données, jamais comme du SQL.
- Les vraies requêtes préparées du serveur sont forcées (`PDO::ATTR_EMULATE_PREPARES => false`) ; les erreurs SQL lèvent une exception (`ERRMODE_EXCEPTION`).
- `LIMIT` et `OFFSET` sont liés en entiers (`PDO::PARAM_INT`).
- Le `WHERE` n'assemble que des morceaux de SQL écrits dans le code ; une valeur saisie n'y est jamais concaténée.
- Une liste d'identifiants (`IN (…)`) reçoit autant de `?` que de valeurs, toutes liées.

```
// PDOSpectacleRepository::rechercher (extrait)
$conditions = [];
$parametres = [];
if ($criteres->date() !== null) {
    $conditions[] = 'so.date_soiree = :date';
    $parametres['date'] = $criteres->date()->format('Y-m-d');
}
if ($criteres->style() !== null) {
    $conditions[] = 'st.slug = :style';
    $parametres['style'] = $criteres->style();
}
$where = $conditions === []
    ? '' : ' WHERE ' . implode(' AND ', $conditions);

$requete = $this->pdo->prepare(
    SpectacleHydrator::SELECT . $where
    . ' ORDER BY … LIMIT :limite OFFSET :decalage');
foreach ($parametres as $nom => $valeur) {
    $requete->bindValue($nom, $valeur, PDO::PARAM_STR);
}
$requete->bindValue('limite', $criteres->taille(), PDO::PARAM_INT);
$requete->bindValue('decalage', $criteres->decalage(), PDO::PARAM_INT);
$requete->execute();
```

> **Audit du code**
>
> Tous les appels à la base de `api/src` ont été relus : chacun passe par `prepare()`, aucun par `query()` ou `exec()` avec une donnée extérieure. Seul le script de données de test (`bin/seed.php`, lancé à la main, jamais exposé sur le web) utilise `exec()` et `query()`, sur du SQL fixe écrit dans le script (liste de tables en dur, comptages).

8.2 Validation des entrées

Avant d'arriver aux cas d'usage, chaque paramètre est contrôlé par `FiltreSpectaclesValidator` ou `IdentifiantValidator`. Une valeur incorrecte est refusée avec un code **400** et un message par champ : elle n'atteint jamais la base.

Paramètre	Règle
`date`	format `AAAA-MM-JJ` et date réelle (`checkdate` : le 31 février est refusé)
`style`	slug de 1 à 60 caractères en `[a-z0-9-]` : ni apostrophe, ni espace, ni point-virgule
`page`, `size`	chiffres uniquement, 6 au maximum ; `page` de 1 à 10 000, `size` de 1 à 50
`{id}`	entier positif de 9 chiffres au maximum (pas de `0x08`, de `8 OR 1=1` ni de nombre géant)
tous	valeur simple obligatoire : un tableau (`?style[]=metal`) est refusé

Les règles métier sont ensuite vérifiées une seconde fois par les entités du domaine (jauge, tarifs, nombre de spectacles), et la base ajoute ses propres contraintes (`CHECK`, clés étrangères, `UNIQUE`).

8.3 Côté site : XSS et en-têtes HTTP

Risque	Protection
Injection de HTML ou de script (XSS)	Handlebars échappe toutes les valeurs (`{{titre}}` : `<` devient `<`) ; la syntaxe sans échappement `{{{ }}}` n'est utilisée nulle part ; hors templates, le texte est inséré avec `textContent` ou `createElement`, jamais avec `innerHTML`
Panier modifié dans le navigateur	chaque ligne relue du `localStorage` est contrôlée (type, catégorie, entiers bornés, 10 billets au maximum par ligne) ; une ligne invalide est ignorée. Les prix du panier sont indicatifs : le serveur les recalculera à la commande (itération 2)
Vidéo piégée	une iframe n'est créée que si l'adresse correspond à `https://www.youtube(-nocookie).com/embed/…` ; la version `nocookie` ne dépose pas de cookie avant la lecture
Redirection vers un autre site	le lien « Retour au programme » mémorisé n'est accepté que s'il commence par `index.html`
Script ou ressource extérieure	en-tête **Content-Security-Policy** : scripts du site seulement, appels réseau vers le site et l'API seulement, iframes YouTube seulement, polices Google seulement, aucun plugin (`object-src 'none'`)
Clickjacking	`frame-ancestors 'none'` et `X-Frame-Options: DENY` : le site ne peut pas être affiché dans un cadre d'un autre site

Risque	Protection
Fichiers mal interprétés, fuite d'adresse	`X-Content-Type-Options: nosniff` ; `Referrer-Policy` (`no-referrer` pour l'API)
Informations sur le serveur	version de nginx masquée (`server_tokens off`), en-tête `X-Powered-By` de PHP retiré (`expose_php = Off`), erreur 500 sans message ni trace en production (`APP_ENV=prod`)
Appels depuis un autre site	CORS : l'API ne renvoie `Access-Control-Allow-Origin` que pour les origines listées dans `CORS_ORIGINS`
Accès à la base, dépendances	MariaDB n'écoute que sur la machine (`127.0.0.1`) ; dépendances sans faille connue (php-jwt 7, la version 6 étant touchée par la CVE-2025-45769 ; Slim ≥ 4.15.2 ; PHPUnit ≥ 11.5.50)

La CSP autorise `'unsafe-eval'` pour les scripts : c'est nécessaire à `Handlebars.compile`, qui transforme les templates en fonctions dans le navigateur. Cela n'ouvre pas de faille XSS puisque les scripts extérieurs et les scripts écrits dans la page (`inline`) restent interdits. Précompiler les templates permettrait de retirer cette exception.

8.4 Tests d'attaque

Les requêtes suivantes ont été envoyées à l'API locale, puis le contenu de la base a été recompté : il est resté identique (37 spectacles, 14 soirées, 42 utilisateurs, 18 838 billets).

Requête envoyée	Réponse
`/spectacles?style=x' OR 1=1 --`	400
`/spectacles?style=metal'; DROP TABLE soirees;--`	400
`/spectacles?date=2027-06-11' OR '1'='1`	400
`/spectacles?date=2027-06-11 UNION SELECT 1`	400
`/spectacles?page=1 OR 1=1` · `?size=10;DELETE FROM billets`	400 · 400
`/spectacles?page=-1` · `?page=99999999999999999999` · `?size=1000`	400 · 400 · 400
`/spectacles?style[]=metal`	400
`/soirees/8 OR 1=1` · `/soirees/1' UNION SELECT * FROM utilisateurs--`	400 · 400
`/soirees/0x08` · `/soirees/9999999999`	400 · 400
`/spectacles/<script>alert(1)</script>`	404
requête `OPTIONS` avec `Origin: https://evil.example`	204 sans en-tête CORS

Côté site, les tests navigateur vérifient aussi qu'un `<img src=x onerror=…>` placé dans l'URL ou dans le panier s'affiche comme du texte, sans être exécuté, et qu'aucune violation de la CSP n'apparaît sur l'ensemble du parcours.

> **Prévu à l'itération 2, avec l'authentification et la commande**
>
> - mots de passe hachés (`password_hash` , bcrypt), jamais stockés ni renvoyés en clair ;
> - jeton JWT signé avec un secret lu dans `.env` , durée de vie courte, rôle administrateur vérifié par un middleware ;
> - prix recalculés par le serveur à partir de la base, sans faire confiance au panier ;
> - places vérifiées et réservées dans une transaction (`SELECT … FOR UPDATE`) pour éviter la survente ;
> - jeton de billet UUID v4 aléatoire, impossible à deviner.

9 Bilan et perspectives

État d'avancement

Étape	Réalisé	Restant
1. Conception	scénarios d'usage, diagramme de classes, base de données et schéma SQL, contrat d'API, maquettes de tous les écrans (ordinateur, téléphone, impression du billet)	—
2. Environnement	dépôt Git, Docker Compose, script SQL, jeu de données de test, structure Sass, squelette de l'architecture hexagonale, README	—
3. Production	**Back** : entités et ports, adaptateurs PDO, actions Slim et routes, conteneur de dépendances, 49 tests unitaires. **Front** : socle Sass et layouts, templates Handlebars et `fetch`, filtrage dynamique et pagination, détail d'une soirée, panier. **Sécurité** : validation bornée, en-têtes HTTP, tests d'attaque	—

La première version opérationnelle couvre donc **F1 à F7, E1, E2 et E4** de bout en bout (base → API → site).

Itération 2

- Authentification : inscription, connexion, mots de passe hachés (bcrypt), jeton JWT, middlewares d'authentification et de rôle administrateur.
- Commande : `POST /commandes` avec middleware de validation ; prix recalculés par le serveur ; Pass 3 jours sur 3 soirées distinctes.
- Paiement : transaction avec verrouillage des soirées (`SELECT … FOR UPDATE`), réservation par `Soiree::reserver()`, création des billets et de leur jeton UUID, annulation complète en cas de refus.
- Billets et backoffice : `GET /billets/{token}`, `GET /commandes/{id}/billets`, `GET /admin/jauges`.
- Front : écrans Connexion, Paiement, Billets imprimables et Backoffice à partir des maquettes ; composition du Pass 3 jours dans le panier.
- Tests unitaires du domaine pour le Pass 3 jours, la commande et le paiement.

Déploiement

Le déploiement sur **docketu.iutnc.univ-lorraine.fr** est préparé : les ports, le mode d'exécution (`APP_ENV=prod`), le secret JWT et les origines autorisées (CORS) sont paramétrés dans le fichier `.env`, et le front détermine l'adresse de l'API à partir de celle du serveur. Les en-têtes de sécurité sont dans la configuration nginx livrée avec le projet, ils s'appliquent donc aussi sur le serveur. La procédure est décrite dans le README.